ATEM ISO ingest — near-real-time, Nextcloud-aware

Goal: get each theatre's ATEM Mini Extreme ISO recordings (up to 9 H.264 streams — 8 camera ISOs + program — at ~10 Mbps each) onto the mothership's Nextcloud as close to real time as the hardware allows, without corrupting anything and without the theatre's uplink falling permanently behind.

The two hardware facts that shape this

- **The ATEM has a built-in FTP server**, and files can be browsed/transferred over it *while recording is still in progress* — confirmed via multiple independent accounts (aaronparecki.com, Directory Opus forum). This is also the only mechanism available — the unit has no NDI output and its Ethernet port doesn't expose the drive over SMB/NFS, only FTP.
- **The files are very likely safe to read mid-write.** Blackmagic markets editing an ISO recording in DaVinci Resolve *before the event even finishes* — that only works if the container format is structured so a partial file is valid up to whatever's been flushed so far (in practice, this means fragmented MP4 — periodic `moof`/`mdat` boxes rather than one index at the very end). Treat this as **very likely true given the marketed capability, but worth a quick empirical check** (copy a file mid-recording, confirm `ffprobe`/a media player can open it) before relying on it operationally.

Why not plain rsync, and why not plain rclone sync

Plain rsync can't connect at all — it needs SSH or an rsync daemon on the far end, and the ATEM only speaks FTP. There's no bridging that directly; the ATEM never exposes rsync or SSH.

A naive periodic `rclone sync`/whole-file re-copy doesn't survive the bandwidth math. Real numbers: at a realistic 4–6 active camera ISOs + program (~10 Mbps each), a 3-hour session is already 50–80 GB. Re-uploading the *entire current file* every 10 minutes would take longer than 10 minutes to transfer at the A-1300's ~170 Mbps ceiling — the sync falls permanently behind almost immediately for any session longer than about 15–20 minutes. Genuinely incremental transfer isn't a nice-to-have here, it's required.

Two ways to get genuinely incremental transfer — both implemented, pick one (see `config/atem-iso-ingest/`):

	Mechanism	Why it's incremental
`pull-iso.py` (default)	FTP's own `REST <offset>` command — the same "resume from byte offset" mechanism `curl --continue-at` / `lftp --continue` / `wget -c` use	Deterministic: explicitly requests only bytes past the last known offset. No cache layer involved.
`rclone-mount-rsync/` (alternative)	`rclone mount` presents the ATEM's FTP share as a local FUSE path; `rsync --append` (not rsync's general checksum-diff algorithm) transfers only the tail past the destination's current size	Also deterministic — `--append` seeks straight to the destination's size rather than reading/hashing the whole file, so it doesn't depend on rclone's VFS cache behaving well for a growing remote file — a real concern for mount-based approaches that `--append` sidesteps entirely

Both are legitimate. `pull-iso.py` has fewer moving parts (no persistent FUSE mounts to supervise); `rclone-mount-rsync/` uses tools you may already operate day to day, at the cost of 12 FUSE mounts to keep healthy. Neither uses rsync's *general* block-checksum algorithm (which genuinely would require reading the whole file every pass) — that algorithm only saves network bytes when both ends speak the rsync wire protocol to each other, which isn't possible against an FTP-only source regardless of what sits in between.

Architecture

Centralized entirely on the Unraid server as its own container (`192.168.1.17`) — no software installed on any theatre laptop, consistent with how VMix/BirdDog Central/NDI Discovery Server/DERP are already consolidated there. Reaches each theatre's ATEM over Tailscale subnet routing (see `docs/tailscale.md` — this is exactly the "route between subnets where needed" case the subnet-router setup was built for). Described below for the default (`pull-iso.py`) — the `rclone-mount-rsync/` alternative differs only in the pull mechanism, writing into the same Nextcloud folders the same way.

```
ATEM (192.168.X.2, FTP) --[Tailscale subnet route]--> Unraid: atem-iso-ingest container
                                                                |
                                                    incremental REST-resume pull
                                                                |
                                                                v
                                              local mirror file, growing in lockstep
                                              (written directly into the folder Nextcloud
                                               has mounted as External Storage — Local)
                                                                |
                                                    targeted `occ files:scan` (short cycle)
                                                                |
                                                                v
                                                  visible in Nextcloud, growing
```

1. **Pull step** (`atem-iso-ingest` container, new — see `config/atem-iso-ingest/`): for each theatre, on a schedule (e.g. every 60–120s), list the ATEM's FTP directory, and for each media file, `REST` -resume from the last recorded byte offset and append the new bytes to a local mirror file. A small state file tracks per-file offsets so a restart doesn't re-pull from scratch.
2. **No separate upload/chunking step.** The pull step writes directly into `/mnt/user/nextcloud-external/TheatreN/ISO/` — the same path mounted into Nextcloud as an External Storage (Local) folder for that theatre. There's no WebDAV re-upload of the growing file at all, so the "whole-file re-sync is too slow" problem simply doesn't apply on this side — it's a same-host filesystem write, not a WAN transfer.
3. **Nextcloud awareness**: a short-interval, *targeted* `occ files:scan --path=...` (not `--all`) picks up the new file size on Nextcloud's side. Because this only touches one theatre's folder and runs locally on the same box, it's cheap even at a 1-2 minute cadence — unlike scanning the whole Nextcloud tree, which would not be.
4. **On session end**, a final pull + final targeted scan catches the last bytes and whatever moov/index finalization happens when the ATEM's recording actually stops.

Second write destination for live editing. The pull step also writes the same growing mirror file to a second mount — the edit suite's dedicated NAS (`192.168.22.x`), not just Nextcloud's External Storage. One read off the ATEM, two writes, not a separate re-sync reading Nextcloud's copy back out. See `docs/live-editing.md` for the full editing subsystem this feeds — not yet reflected in `config/atem-iso-ingest/pull-iso.py` itself, which currently only writes the one Nextcloud destination; adding the second write path is a real code change, tracked in `docs/open-questions.md`.

Bandwidth

Scenario	Aggregate	vs. A-1300 ceiling
Worst case, all 9 streams active	90 Mbps	tight but under ~170 Mbps
Realistic, 4 cams + program	50 Mbps	comfortable headroom
Realistic, 6 cams + program	70 Mbps	comfortable headroom

This rides the theatre's uplink in the *opposite direction* from the incoming SRT feed (~8–50 Mbps, see `docs/open-questions.md`) — if the link is genuinely full-duplex-capable at its rated throughput, those two shouldn't compete much. But the ingest does **not** have the upstream direction to itself: the theatre's ATEM Overseer monitoring stream and Flock SRT preview (~10.4 Mbps each) run theatre → mothership alongside it, plus bursty rclone. The authoritative per-router upstream model — ~129 Mbps worst case against the A-1300's ~170 Mbps ceiling — is in `docs/bandwidth-analysis.md`; the table above covers the ingest in isolation only. Worth validating the full-duplex assumption in practice rather than assuming it, since the ~170 Mbps A-1300 figure was a single-direction benchmark, not a confirmed simultaneous-bidirectional rating.

Master vs. mirror

Frame this as: **the ATEM's own SSD remains the authoritative final master** (complete, guaranteed-valid once recording stops) — pulled in full at the end of each session/day regardless. **Nextcloud holds a near-real-time, continuously-updated mirror** for early review/rough-cut purposes during the event. That framing resolves the tension between "want it fast" and "want it guaranteed correct": you get both, from two different consumers of the same underlying data.

Nextcloud versioning caveat

Nextcloud's versioning app may retain snapshots on repeated changes to the same file path. Configure version retention/expiry for the ISO folders (or disable versioning there specifically) so a multi-hour growing recording doesn't accumulate many near-duplicate historical versions before Nextcloud's own expiry curve catches up. Not a blocker, just worth tuning once this is running for real.

Open items

- **Confirm the 10 Mbps/stream figure and actual active-channel count** against the real ATEM recording settings, rather than relying on the estimate used for the bandwidth math above.

- **Confirm partial-file playability empirically** — copy a file mid-recording, check it opens/scrubs correctly, before treating the "safe to read mid-write" assumption as settled rather than "very likely."
- **Tune the scan interval and pull interval** against real session lengths and available Unraid CPU/disk headroom, once the server's full spec is known (see `docs/open-questions.md`).
- **Confirm the ATEM's actual FTP file/folder naming** (not assumed here — the ingest script lists whatever's present rather than hardcoding filenames, precisely because this wasn't confirmed against a real unit).
- **If using the `rclone-mount-rsync/` alternative**, test `rsync --append` against a genuinely growing file on a real ATEM before trusting it operationally — confirm it only transfers the new tail each pass (e.g. watch network throughput or `--itemize-changes` output), and budget for supervising 12 persistent FUSE mounts (auto-remount on failure), which `pull-iso.py` doesn't need at all.